

profiling ():

⇒ helps

⇒ her

let

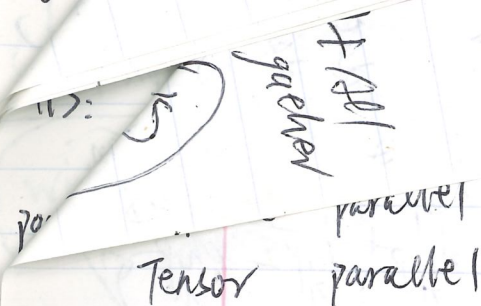
one copies / finished

will

are GPU processes

parallelism primitives

3 important ideas



②: Activation parallelism: manage footprint of memory
sequence parallel ⇒ handle long activation memory for.
long sequence / large batch size / large model

⇒ Naive Data parallelism: split the elements of B sized batch across M machines. Exchange gradients to synchronize

↓
fit Batch of params to various GPUs
↓ Fusion solution ⇒ launch kernel, memo allocation, synchronization, extra time / work spent on managing
memory issues: we need 5 copies of weights and 16 Bytes per param not useful actual computation

↓
ZeRo: solving the memory overhead issue of DP
core idea: split up the expensive parts (state) and use the reduce-scatter equivalence

Zero stage 1: optimizer state sharding

⇒ split up the optimizer state (1st + second moments) across GPUs
everyone has the parameters + gradients

Each worker is responsible for updating a subset of params (corresponding to their slice)

{ gradients ⇒ update parameters ⇒ optimization
optimizer states

Zero stage #1: How it works? { shard gradient → random/random SGD
shard optimizer states across models
same compute/communication cost

step 1: everyone compute full gradient on their subset of the batch

step 2: ReduceScatter the gradients - incur #params communication cost

step 3: Each machine updates their param using their
gradient + optimizer state

step 4: all gather the parameters - incur #params communication cost

Broadcast: One GPU (rank) sends same message or data to all other GPUs

Reduce: combine values from all GPUs (sum, max, etc) and keep the result on one rank (GPU)

All-reduce: combine values from all ranks, then distribute the total result back to every GPU (rank)

Scatter: one GPU splits a dataset and sends a different slice to each GPU